

From Informal Claims to Checked Theorems

An evidence-based history of the IPNPCNS formalization

Subject	How a 60-page interdisciplinary paper became a layered Lean development, and how proof failures, specification boundaries, and audits shaped the result
Primary record	Versioned task logs (Wotan), steering decisions (Project Control), Git states, compiler and build output, source audit, and a checksum-bounded Codex session trace
Final proof corpus	31 Lean modules, 8,367 lines, 439 theorem or lemma declarations, and no admitted proofs or project-specific axioms
Historical stance	Causal reconstruction with explicit evidence classes; not a task diary and not a claim that the final architecture was inevitable
Date	August 2026

IPNPCNS formalization project

A companion to the separate scientific report on the verified mathematics.

Abstract

This report reconstructs how the mathematical claims in a paper on neural population computation were converted into a machine-checked Lean development. Its central thesis is that the work did not proceed by translating formulas in order. It proceeded by repeated *contract refinement*: an informal claim was given an explicit domain, closure convention, quantifier structure, and verification boundary; Lean and mathlib then supplied counterpressure through type errors, missing infrastructure, counterexamples, and proof obligations; the resulting contract was either proved, split, specialized, or retained as a visible premise.

The history is unusually well documented. Git preserves 24 curated checkpoints through the first report. Wotan—a repository-local task queue with one evidence log per task—preserves declared proof contracts and completion evidence. Project Control—a separate ledger of evidence-based steering reviews and owner decisions—preserves why scope changed. A 50 MB, checksum-bounded Codex session prefix also preserves the dense work between commits: full additions and diffs for discarded scratch proofs, compiler diagnostics, theorem-name probes, and migrations into the permanent modules. This record permits a distinction between ordinary elaboration failures, genuine mathematical obstructions, architectural changes of proof, and late verification or typesetting corrections.

Several episodes were decisive. A coordinate-free Moreau/double-rejection pilot validated the general Hilbert-space architecture. Rank deficiency disproved strict coefficient contraction and forced a Fejer/compactness convergence proof. Missing polyhedral-face, Moore–Penrose, and Hilbert–Schmidt APIs led respectively to sufficient active-set certificates, an effective-map pseudoinverse, and a compact Volterra construction by finite-rank approximation. Analytic and probabilistic claims were first isolated behind named interfaces; some were later discharged by Lipschitz, exponential-error, or independent-product theorems, while biological learning premises remained explicit. A final audit against the LaTeX source found no material mathematical discrepancy in the claimed theorem set.

The resulting lesson is methodological rather than celebratory. Fast autonomous formalization remains credible only when theorem contracts, discarded attempts, assumption boundaries, build evidence, and source conformance are all preserved as different forms of evidence.

Contents

1	The history this report tells	1
1.1	A result without inevitability	1
2	Actors, tools, and records	2
2.1	The actors and the workspace	2
2.2	The formal verification toolchain	2
2.3	Wotan and Project Control	2
2.4	How to read the evidence citations	3
3	Evidence and reconstruction	3
3.1	Five kinds of record	3
3.2	The session trace changes the record	3
3.3	Contemporaneous knowledge and later interpretation	4
4	The initial problem and the first bet	4
4.1	A paper broader than a theorem list	4
4.2	The cautious forecast and the compressed calendar	5
5	Contract before proof	5
5.1	Four ambiguities that could not remain informal	5
5.2	Why coordinate-free geometry came first	6
6	The geometric pilot becomes an architecture	6
6.1	Building the missing Moreau stack	6
6.2	What passing the pilot established	7

6.3	Intersection and the first decomposition decision	7
7	Interfaces as epistemic firebreaks	7
8	Expansion by dependency, not by paper order	9
8.1	The first empty queue	9
8.2	Intrinsic subspaces before pseudoinverse notation	9
8.3	The effective-map solution	9
9	Failures at three different levels	10
9.1	Local failures: useful but not profound	10
9.2	Architectural failures: the route changes	11
9.3	A mathematical obstruction: rank deficiency	11
10	The analytic arc: semantics before calculation	11
10.1	Choosing what a signal is	12
10.2	The wavelet normalization false alarm	12
10.3	A concrete compact filter, not a generic slogan	12
10.4	The membrane remainder and the value of a modest constant	12
11	The probabilistic arc: construct the experiment	13
12	Steering as part of the proof method	13
12.1	What Wotan contributed	13
12.2	What Project Control contributed	14
13	The verification standard grew with the project	14
13.1	Verification itself had correction loops	15
14	The source audit as an independent closure gate	15
15	What the session record reveals between commits	16
16	Scale, speed, and what they do not prove	17
17	General lessons for interdisciplinary formalization	17
17.1	Formalize semantic operations before convenient representations	17
17.2	Treat quantifiers and closure as part of the result	17
17.3	Separate exact mathematics, approximation, and empirical realization	18
17.4	Let counterexamples shape the public API	18
17.5	Use interfaces as revisitable hypotheses	18
17.6	Concrete witnesses can be more honest than generic abstractions	18
17.7	Proof logs and proof artifacts answer different questions	18
17.8	Audit the source after the theorem set stabilizes, but before publication	18
17.9	Verification is layered assurance	18
17.10	Autonomous speed increases the value of explicit control	18
18	Residual uncertainty and limits of this history	19
19	Conclusion	19
A	Chronological spine of accepted states	20
B	Evidence ledger	21

1 The history this report tells

There are two legitimate accounts of a formalization, and confusing them weakens both. The companion scientific report asks what the model says, which theorems were proved, which assumptions they require, and what machine checking certifies. This report asks a different question: how did the final theorem set, architecture, and standard of evidence come into being? The distinction was made an explicit closure requirement before the proof phase ended. [E23][E24]

The answer cannot be a list of tasks. A chronological list would show that one file followed another, but not why finite subspaces preceded pseudoinverses, why a strict NNLS contraction bifurcated into two convergence theorems, or why a generic convolution claim became a concrete causal Volterra theorem. The useful unit of history is therefore a *causal development arc*: a claim, the pressure applied to its first formal contract, the decision that followed, and the mathematical object that survived.

Narrative thesis

The formalization advanced through progressive theorem-contract refinement. Its most important results came not only from completing planned proofs but from learning which distinctions the paper’s prose left implicit: pointwise image versus closed conic hull, deterministic implication versus probability theorem, prediction uniqueness versus coefficient uniqueness, a concrete boundary convention versus a generic convolution, and mathematical realization versus biological validity.

This pattern resembles the conjecture–criticism–revision cycle familiar from mathematical practice [7], but here the criticism was partly mechanized. Lean did not decide research scope. It did make type, domain, dependency, and logical gaps impossible to ignore. Project control supplied the complementary decision layer: whether a gap should be repaired, exposed as a premise, decomposed, or left outside the theorem set. [E02][E07][E10][E14]

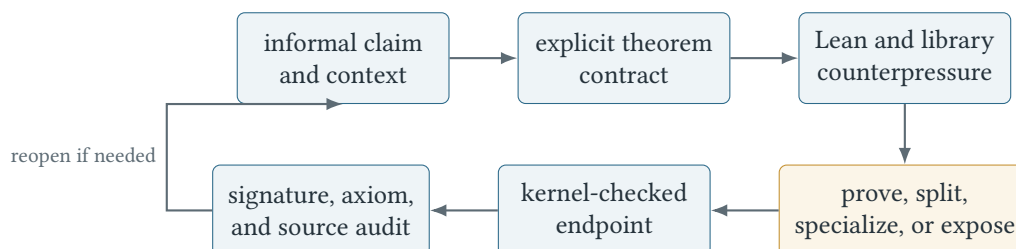


Figure 1: The recurring development cycle. The arrows describe logical counterpressure and revision, not a rigid procedural sequence.

1.1 A result without inevitability

The final tree is clean: named modules, stable public endpoints, a complete build, and a source audit. That cleanliness is an endpoint property. It should not be read backward as evidence that the path was direct. The session trace records hundreds of intermediate patches, temporary theorem probes, failed rewrites, typeclass problems, and deleted scratch files. Several permanent modules were literally moved from multi-stage scratch developments only after their statements and proof routes had stabilized. [E26]

Nor did every failure have equal significance. An unavailable theorem name is an API lookup problem. A deterministic heartbeat timeout may indicate an elaboration shape that should be redesigned. A nonzero kernel vector fixed by an iteration is a mathematical counterexample to strict contraction. The history becomes informative only when these are separated.

2 Actors, tools, and records

The names used in this history describe a small project environment, not knowledge the reader is expected to bring. The environment had one human owner, one continuing AI-agent task, one Git repository, a pinned Lean toolchain, and two lightweight repository-local control mechanisms. None of the control mechanisms participates in Lean’s logical trust chain.

2.1 The actors and the workspace

The *owner* supplied the paper, set the scientific and publication goals, authorized continuous autonomous execution, resolved matters of scope and external publication, and challenged an early claim about missing historical evidence. The *agent* was Codex, an AI system operating through a continuing task. It could read and patch files, run shell commands, compile Lean and LaTeX, inspect rendered pages, and use Git and GitHub under the owner’s authorization.

The *repository* was the ordinary filesystem directory containing the paper and, eventually, all Lean source, reports, evidence records, and Git metadata. Git stored content-addressed snapshots called commits. A clean commit is therefore evidence of an accepted file state and its relationship to earlier states; it is not a record of every attempt made before that state was accepted.

The phrase *session trace* refers to the JSON Lines event log written by Codex under the user’s home directory, conventionally abbreviated `~/ .codex/`. Each line records an observable event such as a message, command, patch, or tool result. The raw trace remained private and outside Git. A repository document records the hash and cutoff of the prefix used here, so the historical sample is reproducible without publishing the entire operational log.

2.2 The formal verification toolchain

Lean is an interactive theorem prover. Mathematical definitions and theorem statements are written in Lean source files; tactics construct proof terms; a small kernel checks the fully elaborated terms. *mathlib* is the imported, machine-checked mathematical library providing such foundations as Hilbert spaces, convexity, matrices, measure theory, and finite combinatorics. *Lake* is Lean’s project and build tool: it resolves the pinned dependency graph and asks Lean to check the modules.

These roles matter historically. A compiler diagnostic can refute one attempted term or reveal a missing library interface. A successful clean Lake build establishes that the accepted modules elaborate and kernel-check together. Neither result alone shows that the theorem statement faithfully represents the paper. That separate question is addressed by the source audit.

2.3 Wotan and Project Control

Wotan was the execution ledger. Its machine-readable file `wotan/backlog.json` stored task identifiers, state, size, and dependencies. For each task, `wotan/dev-log/T-NNNN.md` recorded the mathematical context, acceptance criteria, planned verification, implementation notes, obstacles, evidence, and outcome. Only one task was active at a time. A task could not be called complete until its status, evidence log, and Git commit agreed.

Project Control was the steering ledger, stored in `PROJECT-CONTROL.md`. It was invoked at scientific gates, after major results or falsifications, and when the executable queue became empty. A review compared the current objective, evidence, assumptions, uncertainty, and remaining value, then recommended continuing, redirecting, evaluating, preserving, waiting, or stopping. Decisions requiring authority were recorded separately as owner decisions and only then translated into Wotan tasks.

The division can be summarized as follows:

Mechanism	Question answered	What it cannot establish
Wotan	What concrete work is ready, active, blocked, or complete, and what evidence closes it?	Whether the overall scientific direction is still the best use of effort
Project Control	Given current evidence and uncertainty, should scope continue, change, close, or await an owner decision?	The truth of a Lean theorem or the empirical adequacy of the CNS model
Lean kernel	Does the elaborated proof term have the exact theorem type under the recorded assumptions?	Whether the formal contract is the intended reading of the paper
Source audit	Does the formal contract agree with the identified PDF and LaTeX source?	Whether biological systems satisfy the model assumptions

Wotan and Project Control are project-specific names for transparent files and rules, not external databases or invisible services. A reader needs no installation of either system: the relevant records are ordinary versioned JSON and Markdown. Later sections ask what these mechanisms contributed to the result, but their definitions and evidentiary limits are fixed here.

2.4 How to read the evidence citations

Blue bracketed identifiers such as [E01] refer to the evidence ledger in section B. The ledger points to task logs, decisions, commits, audits, build records, and the bounded session trace. Files shown with a typewriter face are repository paths; a leading `~/` means the user's home directory. Commit identifiers denote Git snapshots. The terms *contemporaneous* and *retrospective* distinguish evidence recorded during proof work from later analysis of the completed corpus.

3 Evidence and reconstruction

3.1 Five kinds of record

The reconstruction uses five provenance classes. No single one is treated as a complete account.

The evidence identifiers in the margin-like blue brackets refer to the ledger in section B. That ledger, in turn, points to the repository artifacts and the separately bounded session audit. Material historical propositions in this report are tied to at least one such identifier.

3.2 The session trace changes the record

An initial evidence review contained an overly cautious statement: no record was known to preserve every abandoned tactic or scratch proof. The owner pointed to the full Codex JSONL session under `~/codex/`. Inspection changed the historiographical conclusion. [E26]

The audited prefix contains 427 successful patch-completion events. Of these, 174 changes concern 33 scratch, probe, or axiom-audit files: 34 additions, 128 updates, and 12 deletions. Added files carry full content; updates carry unified diffs; command outputs retain the Lean diagnostics. The 33 patches to `ScratchT17.lean` and 26 patches to `ScratchT18.lean`, for example, survive even though those paths do not exist in the final tree. The trace also records their moves into the wavelet and Volterra modules. [E26]

Table 1: Historical evidence classes and their proper use.

Class	Record	What it supports
C	Contemporaneous task record	Declared context, acceptance criteria, implementation summary, obstacle, and verification evidence at task closeout.
D	Steering decision	What evidence and uncertainty were considered at a gate, and why the next direction was selected.
G	Git state	Durable file content, order of curated checkpoints, commit timestamps, and the diff between accepted states.
R	Retrospective audit	Later source conformance, coverage classification, trust review, and report QA. It corroborates the final state but must not be projected backward as prior knowledge.
S	Session trace	Tool-mediated patches, scratch evolution, compiler feedback, and contemporaneous updates between commits. It does not reveal unexpressed cognition.

Session-trace boundary

The main session prefix is fixed by a SHA-256 digest and a 6,276-record cutoff. The audit reads observable messages, tool calls, patches, and outputs. It deliberately does not reproduce model-internal reasoning records or copy the raw 50 MB log into the repository. Thus the report can reconstruct expressed proof search without turning a private operational log into an indiscriminate appendix.

This evidence does not make Git redundant. The trace shows what was tried; Git shows what was accepted. Wotan explains the intended contract; Project Control explains why work changed direction. A compiler error proves only that one intermediate term failed. A later successful build and axiom audit establish the status of the accepted endpoint.

3.3 Contemporaneous knowledge and later interpretation

Dates matter because the LaTeX source audit occurred after proof expansion closed. Early work used the rendered PDF, text extraction, visual inspection, and local mathlib experiments. The later source audit confirmed the closure convention, polar sign, quantifiers, constants, and qualifications, but that confirmation was not available to guide the first proofs. [E01][E23]

Accordingly, phrases such as “the source confirmed” refer to retrospective corroboration. Phrases such as “the task record identified” refer to evidence available at the time. Interpretive claims – for example, that interfaces acted as epistemic firebreaks – are explicitly presented as conclusions drawn from multiple records, not as quotations from any one log.

4 The initial problem and the first bet

4.1 A paper broader than a theorem list

The repository began with a 60-page PDF and no Git or Wotan metadata. The paper mixed linear-subspace geometry, closed convex cones, finite probability, temporal signal analysis, nonnegative least squares, approximate embeddings, and a biological interpretation. Its central mathematical contributions appeared mainly as numbered equations and proof blocks rather than a conventional

theorem hierarchy. The first work therefore mapped equations, definitions, qualifications, and dependencies rather than searching only for labels such as “Theorem.” [E01][E26]

The feasibility review separated three questions:

1. Do the mathematical conclusions follow from precise assumptions?
2. Do idealized learning and population interfaces imply the claimed cone operations?
3. Do actual neural populations satisfy those interfaces?

Lean could address the first and conditional forms of the second. The third remained empirical. This separation was not a final-report disclaimer added after success; it was part of the first task contract and shaped the module boundary between general mathematics and `Model/Population.lean`. [E01][E05]

4.2 The cautious forecast and the compressed calendar

The initial report estimated four to eight person-weeks for a human Moreau and double-rejection pilot and twelve to twenty-four person-months for broad Sections 2–5 coverage. Minutes later, the owner clarified that an autonomous AI agent was to perform the work and expected hours rather than human project time. The response reframed the four proposed work packages as Wotan idea tasks and selected the pilot as the first decisive test. [E01][E26]

The Git record from the feasibility commit through the source-audit commit spans about five hours on 3 August 2026; the first scientific-report commit followed less than half an hour later. This is a fact about preserved timestamps, not a controlled productivity comparison. The original estimate assumed an experienced human, maintainable mathlib-based work, and conventional review. The realized project used an autonomous agent, aggressive scratch prototyping, a fixed local environment, and a publication-oriented but repo-local theorem scope. It did not upstream a general library or validate biology. [E01][E25][E26]

The first bet

Instead of attempting the paper in order, the project chose one identity that exercised all foundational decisions: metric projection, the source’s non-positive polar, Moreau decomposition, closure-safe cone operations, and the passage from a pointwise equality to equality of closed conic hulls. If that identity failed under the paper’s quantification, the architecture would be reconsidered before broader coverage. [E01][E02]

5 Contract before proof

5.1 Four ambiguities that could not remain informal

The first permanent Lean code was preceded by `FORMALIZATION-SPEC.md`. It fixed conventions whose omission would have allowed an apparently successful proof to mean something different from the paper. [E02]

The Lean and mathlib versions were pinned at the same gate. Mathlib’s Hilbert projection theorem supplied existence of a minimizer, but no bundled cone-specific Moreau API supplied the paper’s exact bridge. A local probe demonstrated that a noncomputable metric projection with membership and minimality could compile. The session trace preserves even the correction of one mistaken singular-value API name inside that probe. [E01][E02][E26]

Table 2: The initial theorem contract and its downstream consequences.

Issue	Frozen interpretation	Why it mattered later
Closure	Every arbitrary conic hull, sum, and image used by the cone algebra is closed in the norm topology.	Projected generator sets could not be treated as if their pointwise image were already a closed cone; absorption and intersection required explicit hull arguments.
Polar sign	The source polar is $C^\circ = \{z : \langle z, y \rangle \leq 0 \text{ for all } y \in C\}$, opposite in sign to mathlib’s ordinary dual-cone orientation.	Moreau residuals, reflection laws, and rejection formulas could share one paper-facing membership theorem without sign drift.
Quantification	The cone core uses arbitrary real Hilbert spaces and arbitrary closed convex cones; finite generation belongs to a separate population layer.	The headline identities were not weakened to matrix or polyhedral special cases, while finite active faces could later be introduced without contaminating the core.
Image equality	A pointwise projection identity and equality of their closed conic hulls are separate proof obligations.	The double-rejection theorem first identifies individual projections and only then lifts the result extensionally through Conic.

5.2 Why coordinate-free geometry came first

The specification deliberately avoided matrices in the core. Metric projection, polarity, and orthogonality are the mathematical content of Moreau decomposition; matrix coordinates are one representation. This choice later paid twice. The subspace theory could be proved before a Moore–Penrose inverse existed, and the finite population cones could live inside infinite-dimensional signal Hilbert spaces because only their spans needed finite dimensionality. [E02][E09][E15][E19]

This was not merely a preference for abstraction. It was a dependency decision. Had the pilot been expressed through pseudoinverse matrices, the absence of a mathlib Moore–Penrose API would have blocked the first gate for a reason unrelated to the headline cone identity.

6 The geometric pilot becomes an architecture

6.1 Building the missing Moreau stack

The pilot introduced a chosen metric projection, its variational characterization, the polar residual, projection–residual orthogonality, Moreau decomposition, bipolarity, and closure-safe conic projection and rejection. The final double-rejection proof followed the paper’s semantic architecture: establish a pointwise orthogonal cone–polar decomposition, use Moreau uniqueness to identify the second projection, and only then take closed conic hulls. [E03] This construction is the project-specific formal counterpart of the classical polar-cone decomposition [2, 4].

The session trace shows a denser path than the commit. A failed rewrite in the metric projection module, an unavailable local theorem identifier, a failed linear-arithmetic closure, and an application mismatch in the operation layer occurred within roughly three minutes. These were local elaboration and API repairs. They did not force a change in the mathematical proof plan, which was already fixed by the contract. [E26]

A separate infrastructure failure did occur: dependency materialization was interrupted by restricted network access. The recovery used the installed, version-matched mathlib cache and obtained only the missing pinned dependency. The resulting project retained a reproducible version manifest. The distinction is important: network recovery changed how dependencies were materialized, not what the theorem asserted. [E03][E26]

6.2 What passing the pilot established

The pilot did more than prove one equation. It demonstrated that:

- the source polar convention could be isolated cleanly;
- mathlib’s Hilbert projection theorem was sufficient foundation for a local Moreau theory;
- arbitrary closed-cone quantification was viable;
- closure bookkeeping was manageable without hidden image-closedness assumptions; and
- the accepted endpoints could be built without `sorry` or custom axioms and audited at the declaration level.

These were the reasons the project continued, not merely the speed with which the first commit appeared. Project control recorded the next gate as the much longer intersection identity. [E03][E04]

6.3 Intersection and the first decomposition decision

Before formalizing the intersection identity, the work considered whether polar algebra might yield a shorter proof. The rejection-only form still required the paper’s four Moreau decompositions and cross-term control, so the direct geometry was retained. One large theorem needed an increased heartbeat allowance, but the kernel-checked term added no assumptions. [E04]

At the same time, the approximate-invariance section was divided. The deterministic projection-error implication was formalized with explicit relevant-vector and transformed-optimality premises. The polarization step and sparse Gram bound were independent and became a separate task. The random Johnson–Lindenstrauss guarantee remained outside the deterministic theorem. This was the first repeated use of a pattern that later dominated the project: prove the implication whose assumptions are precise, name the missing bridge, and keep probability distinct from geometry. [E04][E06]

7 Interfaces as epistemic firebreaks

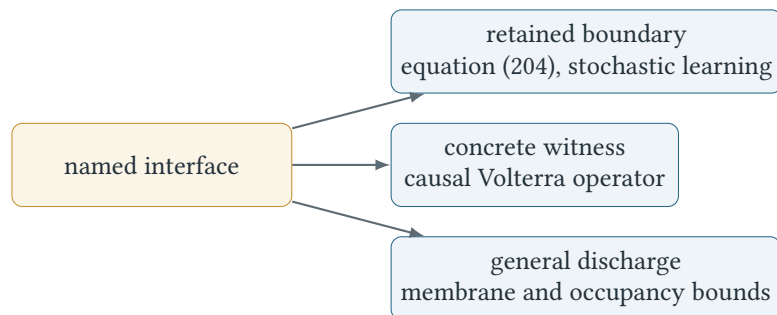
An explicit premise is sometimes mistaken for a failed formalization. In this project, named interfaces had a more disciplined role. They prevented the proof assistant from converting underspecified prose into hidden axioms while allowing valid downstream consequences to be checked. Some interfaces remained model boundaries; others were later discharged when a precise witness became available.

The population layer made the boundary especially clear. Finite NNLS fixed points and the error implications were provable. Support selection, learned approximation, and biological adequacy were not. Recording the latter in `MODEL-OBLIGATIONS.md` meant that later progress could target one obligation without pretending the whole interface had been derived. [E05]

The historical lesson is that an interface is useful only if it is visible, source-shaped, and revisitable. A structure field whose name and contract match the paper’s approximation can be audited. A definition that simply says a population “implements projection” cannot.

Table 3: Representative interface life cycles.

Claim	Initial formal treatment	Later status
Population approximation (204)	Named support-selected approximation premise in <code>PopulationRegion</code> .	Remains a model premise; only (206)–(208) and zero-error hull consequences are proved. No learning or biological dynamics is invented.
Stochastic NNLS convergence	Recorded as an obligation because “standard assumptions” supplies no filtration, sampling law, or moment bounds.	Remains outside scope; deterministic batch convergence was developed instead.
Active-face formula (199)	Initially a field of the population interface.	Discharged by a finite sufficient KKT/Moreau certificate; necessity and a full relative-interior partition remain unclaimed.
Compact convolution	Boundary convention, finite-rank approximation, and compactness were separate named premises.	Discharged for a concrete nonzero causal unit-step Volterra operator, not for every kernel or boundary convention.
Membrane $O(\Delta t^2)$ term	A named <code>QuadraticRemainder</code> interface.	Discharged under an explicit Lipschitz drive with bound $(L/\tau) \Delta t ^2$.
Exponential occupancy approximation	Represented by a named error-bound interface after the exact finite expectation.	Discharged constructively for $0 < p \leq n$, with signed and absolute error bounds.
Two-message overlap	Conditional on uniform marginals and targetwise independence.	Exact for a later Cartesian-product experiment; still not a statement that biological messages are independent.

**Figure 2:** A named interface had three legitimate outcomes. Remaining explicit was not an admission of proof; specialization and discharge required new mathematics.

8 Expansion by dependency, not by paper order

8.1 The first empty queue

The initial four tasks ended with a verified cone core, deterministic approximation interfaces, finite NNLS fixed-point mathematics, and conditional population consequences. The queue could have stopped there. Instead, a direction review compared the achieved results with the paper-wide inventory and selected finite subspaces as the next shared foundation. This was the first expansion decision made after the owner’s instruction to continue autonomously. [E08][E09]

The chosen order did not follow the article. It followed dependency and risk:

1. establish intrinsic finite-subspace operations and comparison measures;
2. derive the NNLS step-size theorem from Gram bounds;
3. close exact cone and circuit laws;
4. formalize cone similarity and its finite-frame proxy;
5. construct the exact finite occupancy experiment; and
6. only then select the signal-space semantics needed by several analytic examples.

This order allowed each task to leave a reusable interface for the next without forcing an unresolved representation detail onto the critical path. [E09][E10][E11][E12][E13][E14]

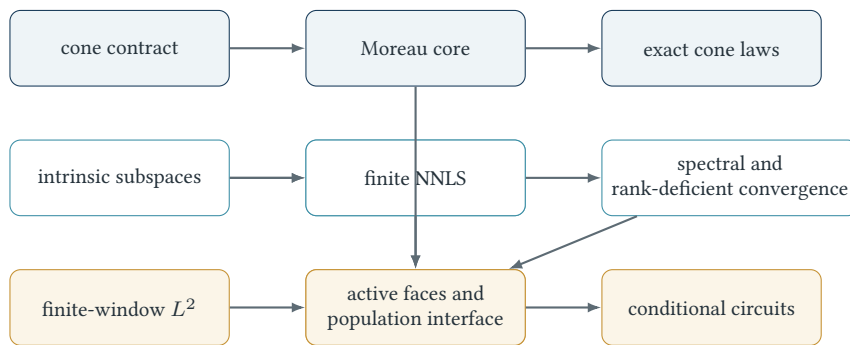


Figure 3: A simplified dependency view. The historical sequence followed shared foundations and exposed gaps rather than the linear order of paper sections.

8.2 Intrinsic subspaces before pseudoinverse notation

The subspace task proved the coordinate-free content of equations (12)–(23), the range identity $\text{Col}(AA^T) = \text{Col}(A)$, comparison formulas, and a concrete failure of distributivity. Mathlib lacked the general Moore–Penrose API needed for the paper’s AA^+ notation. Because none of those intrinsic results depended on that notation, Project Control preserved the gap as a later task rather than blocking the geometry. [E09]

This split prevented two common distortions. The project did not assume a pseudoinverse axiom merely to match notation, and it did not weaken the paper to full-rank matrices. When the representation task was later executed, the construction covered arbitrary rectangular, zero, and rank-deficient matrices. [E15]

8.3 The effective-map solution

The final pseudoinverse did not use a full singular-value decomposition. It restricted a linear map to the orthogonal complement of its kernel, corestricted it to its range, proved the resulting effective

map bijective, and inverted that equivalence. The two products became the orthogonal projectors onto the range and kernel complement; the four Penrose equations and uniqueness followed. [E15]

The session record shows 25 successive patches to the permanent module. Several failed rewrites were not failures of linear algebra but consequences of dependent projection instances: equality of subspaces did not make the typeclass witnesses definitionally identical. An extensional projector-equality lemma became the durable bridge. [E26]

This episode illustrates a recurring distinction. A coordinate-free theorem may be mathematically simpler and more reusable, while a source-faithful coordinate formula still has publication value. The appropriate architecture proves the former first and derives the latter as a representation theorem.

9 Failures at three different levels

The polished Git history makes nearly every task appear to go from specification to success in one commit. The session trace shows that this is a curated fiction useful for version control but inadequate for history. Yet merely counting compiler errors would be equally misleading. The failures fall into three levels.

Table 4: Failure classes and their effects on the final development.

Level	Typical evidence	Proper response
Elaboration and API	Unknown theorem name, failed rewrite, coercion mismatch, stuck typeclass inference, or a simplifier that sees the wrong normal form.	Probe the library, expose intermediate expressions, replace a brittle tactic, or add an extensional helper. The theorem contract normally remains unchanged.
Proof architecture	Heartbeat timeout, unavailable abstraction, or a route that makes later obligations intractable.	Change the mathematical decomposition: use an effective map instead of SVD, direct quadratic-form counting instead of an eigenvalue route, or simple-function density instead of a custom time partition.
Theorem contract	A counterexample, missing hypothesis, ambiguous boundary convention, or empirical premise.	Split the theorem, add the smallest visible premise, select a concrete model, or leave the claim outside scope. Never disguise the repair as tactic work.

9.1 Local failures: useful but not profound

The sparse Gram prototype encountered unavailable summation lemmas, coercion problems, and failed congruence applications over twelve patches. The NNLS prototype encountered missing derivative combinators and rewrite failures over thirteen patches. The wavelet scratch file accumulated 33 patches while interval integrals, trigonometric normal forms, L^2 representatives, and finite examples were made to agree. These sequences matter because they show the actual cost of formal proof, but none by itself refuted the source mathematics. [E06][E08][E17][E26]

A better historical account records what those errors caused. In the Gram proof, they encouraged an explicit expansion of diagonal and ordered off-diagonal terms, which made the exact $k-1$ count visible. In NNLS, they encouraged proving one finite loss-increment identity from which both derivative and optimality results could be derived. In the wavelet development, they produced reusable integer-mode integral lemmas rather than a collection of numerical checks.

9.2 Architectural failures: the route changes

The active-face problem began with the paper’s intrinsic language of faces and relative interiors. Mathlib did not expose a matching polyhedral-face API. Rather than build a full face lattice, the project formalized the exact sufficient conditions used by the projection: a nonnegative active coefficient expansion, polar inequalities against every generator, and orthogonality. Moreau uniqueness then certified the selected span projection. Boundary certificates were allowed to overlap, with equality of the projected point proved. This certificate-centered view is consonant with the classical active-set treatment of nonnegative least squares [3]. [E07]

This final theorem is both stronger and weaker than an informal equation (199). It is stronger as a machine-checkable certificate: every finite inequality is visible and degenerate active sets are covered. It is weaker as a classification: necessity and a full relative-interior partition are not claimed. Calling this a “workaround” would miss the methodological point. It replaced an unavailable global theory with the local theorem actually needed by the population interface.

The compact-convolution proof underwent an even clearer route change. A direct finite-partition construction generated difficult elaboration and a deterministic heartbeat timeout in `ScratchT18.lean`. The surviving proof introduced a general map from an L^2 Hilbert-valued kernel to a bounded analysis operator, proved that indicator-constant kernels give rank-one operators, used density of simple functions, and concluded compactness by norm closure. This was more general than the planned special partition while remaining concrete about causal boundary conditions. [E18][E26]

9.3 A mathematical obstruction: rank deficiency

The most important failure was not a Lean error. For the projected batch NNLS map, the familiar step condition $0 < \varepsilon < 2/L$ does not imply a strict contraction on coefficient space when the prediction map has a nontrivial kernel. A nonzero kernel direction is fixed by the unprojected linear error map, so no factor $q < 1$ can contract every coefficient difference. The project proved this obstruction instead of hiding it behind a lower singular-value assumption. [E10]

Two honest theorems replaced the hoped-for single statement:

- under positive lower and upper spectral bounds, strict contraction gives a unique coefficient minimizer and geometric convergence;
- with only an upper bound and a nonempty minimizer set, Fejer descent, summable prediction error, finite-dimensional compactness, and a cluster minimizer give convergence to *some* coefficient minimizer. All minimizers share the same prediction, while coefficients need not be unique.

The second proof was not a weakened version of the first. It was a different proof architecture forced by the geometry of the kernel. Project control recorded this as a redirection, and a later task completed the Fejer/compactness argument without a positive lower spectral bound. [E10][E16]

Counterexamples are architectural evidence

The kernel obstruction did more than close a false route. It determined the public API: prediction uniqueness and coefficient uniqueness became separate theorems, and the rank-deficient convergence module could not accidentally acquire a full-rank hypothesis. [E10][E16]

10 The analytic arc: semantics before calculation

10.1 Choosing what a signal is

The signal layer could not begin from a convolution formula alone. It first selected Lebesgue measure restricted to $[0, T]$ and Bochner L^2 classes for scalar and finite-vector signals. This decision immediately corrected two informal habits. Pointwise matrix identities had to be stated almost everywhere because L^2 elements are equivalence classes. And the image of a closed cone under a bounded linear map could not simply be declared closed; the formal cone image used a closed conic hull. [E14]

The first task at this layer stopped deliberately at named interfaces for boundary conditions, generic convolution realization, compactness, Laplace transfer, and the quadratic membrane remainder. Exact linearity, norm bounds, conic aggregation, and finite-dictionary facts were proved. This left a coherent theorem layer without pretending that unspecified analytic hypotheses had been supplied by notation. [E14]

10.2 The wavelet normalization false alarm

During the initial PDF review, extracted text made the normalization factor for the three Hann-windowed wavelets look suspicious. The session update records a genuine concern that the displayed constant might not give norm one. High-resolution inspection of the rendered formula resolved the scope of the square root, and later exact integration proved the source constant correct. [E01][E17][E26]

This episode is useful because it shows two distinct error channels. PDF text extraction can corrupt notation; a numerical or symbolic plausibility check can then raise the right question without settling it. The final answer required an exact Lean proof in the chosen recording-window measure.

The proof history was substantial. `ScratchT17.lean` first probed missing integral theorem names, then derived integer sine and cosine integrals, product-to-sum identities, Hann-square expansions, means, Gram entries, and L^2 inner products. A deterministic elaboration timeout and many local mismatches were resolved before the file was renamed and moved into `IPNPCNS/Examples/OrthonormalWavelets.lean`. Its final theorems also checked three NNLS examples and kept baseline subtraction separate from literal nonnegative firing rates. [E17][E26]

10.3 A concrete compact filter, not a generic slogan

The source’s compact-filter discussion did not fix every kernel extension or boundary convention. The formalization chose a nonzero unit-step causal response on the finite window, giving the Volterra operator

$$(Vx)(t) = \int_0^t x(s) ds.$$

The final module proves its almost-everywhere integral formula, the bound $\|V\| \leq T$, finite-rank approximation in operator norm, and compactness. It separately proves that the chosen realization is not periodic and is not the alternative whole-window zero-extension formula. [E18]

The missing Hilbert–Schmidt abstraction therefore did not become an axiom. It became a prompt to prove compactness from simpler certified pieces. The resulting theorem is narrower in kernel scope but stronger in semantic clarity than an unqualified claim about “the convolution operator.”

10.4 The membrane remainder and the value of a modest constant

Equation (33)’s $O(\Delta t^2)$ term originally remained behind a named `QuadraticRemainder`. An empty-queue review later selected it as a bounded analytic gap. The exact first-order low-pass step was written as an exponentially weighted Bochner integral. Under a Lipschitz premise on the net drive, subtracting the frozen initial drive produced an explicit remainder bounded by $(L/\tau)|\Delta t|^2$. [E20]

A sharper elementary factor of one half was considered. The final proof used the uniform interval bound instead, because it supplied the required step-independent quadratic constant with a smaller and more auditable argument. This is a small but representative scientific choice: formal verification need not optimize every constant when the claimed order and interface are preserved, but it must say which constant was actually proved. [E20][E26]

11 The probabilistic arc: construct the experiment

The occupancy formula after equation (30) appears elementary, but its meaning depends on a sampling model. The formal experiment made each of m axons choose uniformly from the p -element subsets of n targets. Choices are without replacement within one axon and independent between axons. Treating all mp collaterals as independent target draws would have proved a different formula and was explicitly rejected. [E13]

The exact expected occupied-target count followed by indicator expansion and finite product averaging. Zero targets, zero axons, zero collaterals, and the impossible case $p > n$ received separate statements. The source's exponential replacement and two-message overlap were initially left behind different interfaces because they used different extra assumptions. [E13]

Later empty-queue reviews returned to both interfaces. For $0 < n$ and $p \leq n$, elementary exponential remainder and finite power-difference bounds proved that the exponential approximation is a lower bound with absolute count error at most mp^2/n . This was a genuine discharge of an error interface. [E21]

The overlap statement required a different move. Two messages were sampled from the literal Cartesian product of the exact one-message sample space. Targetwise independence then followed from the construction, and expected overlap became exactly q^2/n , where q is the exact one-message expectation. Independence was made structural in the selected mathematical experiment; it was not inferred for neural messages. [E22]

The session trace reveals the final proof repair. Directly unfolding the structure predicates failed. Changing the goals to the literal finite-expectation equalities made the product factorization visible, after which the existing abstract overlap theorem could be instantiated. [E26]

12 Steering as part of the proof method

Wotan and Project Control are subjects of this report because they influenced which mathematics was attempted. They are not presented as scientific evidence for the theorems themselves. Lean's kernel and the audited source tree play that role.

12.1 What Wotan contributed

Each task combined a claim-oriented context, acceptance criteria, hard verification endpoints, manual checks, risks, an approach, implementation notes, obstacles, evidence, and an outcome. The initial continuations were stored as inactive ideas. After the owner authorized continuous autonomous work, Project Control promoted them into an executable queue and permitted creation of follow-on tasks. [E01][E02]

This mattered most at boundaries. The sparse Gram proof was split from the already verified projection-error theorem. Active-face construction and deterministic NNLS convergence were separated from the conditional population interface. The missing pseudoinverse notation was preserved without delaying intrinsic subspaces or NNLS. The source audit and two reports were blocked behind a moving proof-phase barrier so that they would describe a stable theorem set. [E04][E05][E09][E23]

The logs also make negative scope durable. A theorem not attempted because its stochastic assumptions were absent does not silently disappear; it remains in `MODEL-OBLIGATIONS.md` and in the task outcome. This is especially important for interdisciplinary work, where an omitted empirical bridge can otherwise be mistaken for a proved consequence.

12.2 What Project Control contributed

Project-control reviews occurred at scientific gates and whenever the actionable queue became empty. Each review recorded the current evidence, uncertainty, selected direction, and revisit condition. The pattern was not “always continue.” It included preserve, evaluate, redirect, and eventually close proof expansion in favor of source conformance. [E03][E09][E18][E22][E23]

Three types of steering decision were especially productive:

Protect the critical path.

Prove coordinate-free geometry before pseudoinverse representation; separate the already valid deterministic implication from an independent probability theorem.

Change proof architecture when the mathematics demands it.

Preserve the full-rank contraction theorem and create a distinct rank-deficient Fejer task after the kernel obstruction.

Reopen named boundaries selectively.

When the queue emptied, choose a compact, source-shaped obligation such as span reduction, the membrane remainder, or occupancy error rather than expanding into an underspecified stochastic or biological program.

The final redirection is as important as the expansions. After the independent two-message theorem, the remaining open areas required a new stochastic or biological specification, generic transform theory, or a much larger intrinsic face development. Project control judged source fidelity more limiting than another small proof and released the LaTeX audit. [E22][E23]

Autonomy requires visible stopping logic

Continuous autonomous execution did not mean that every adjacent mathematical topic became authorized scope. The combination of a maintained obligation register, an empty-queue review, and explicit revisit conditions supplied a reasoned boundary between productive continuation and unbounded theorem accumulation.

13 The verification standard grew with the project

“It compiles” meant different things at different moments. The first feasibility probe established only that mathlib could support a chosen metric projection. The final closure claim combined a complete build, proof-gap and unsafe-declaration scans, public theorem-signature review, a 38-endpoint axiom audit, equation-level source conformance, and report artifact QA. Treating these checks as interchangeable would erase how the standard of evidence matured. [E01][E03][E23][E24]

Lean’s small trusted kernel checks elaborated proof terms; mathlib provides previously checked definitions and theorems [5, 6]. The project adds no custom axiom. The representative endpoints report only `propext`, `Classical.choice`, and `Quot.sound`, the accepted standard principles used by the chosen Lean/mathlib development. This audit does not mean that the code is constructive or computational: metric projection and several finite-dimensional choices are intentionally noncomputable. [E23]

Table 5: Layers of evidence added during the project.

Layer	Check	Question answered
1	Focused probe or scratch compilation	Does the proposed type, library bridge, or proof route elaborate in the pinned environment?
2	Focused module build	Does the accepted module compile with its actual imports and namespace?
3	Complete lake build	Does the whole dependency graph elaborate together? The final build completed 2,960 jobs.
4	Gap and declaration scan	Do project sources contain sorry, a custom axiom, or an unsafe declaration? The final scans found none.
5	Signature and #print axioms audit	Does the public theorem quantify as intended, and on which logical principles does the compiled constant depend?
6	LaTeX source conformance	Were signs, constants, domains, quantifiers, equation references, and nearby qualifications read correctly from the authoritative source?
7	PDF extraction and visual QA	Does the reader-facing artifact contain the intended text, metadata, tables, figures, and pagination without visual defects?

13.1 Verification itself had correction loops

The audit process was not exempt from failure. A task-level axiom check once ran in parallel before the new object files existed. The source-wide audit similarly raced the full build and initially reported a missing wavelet object file. Five temporary signal endpoint names were also misspelled in the first audit script. Sequential reruns with corrected names passed; no project theorem changed. [E05][E23][E26]

These incidents justify preserving commands and outputs rather than recording only “audit passed.” They also show why verification evidence should be reproducible and why a failed audit command must be classified before it is interpreted as a failed theorem.

The build-job count grew from 2,356 at the foundational specification to 2,960 at proof closure. The number is an environment-specific Lake build count, not a direct measure of project lines or theorem difficulty. Its historical value is that every closed task rechecked an expanding dependency graph rather than compiling only the new file. [E02][E23]

14 The source audit as an independent closure gate

The proof work initially relied on the distributed PDF, extracted text, rendered pages, and manual cross-checks. The owner later required a stronger gate: after proof expansion was judged complete, inspect the author-supplied LaTeX archive, reopen any affected proof, and only then write the two reports. Project control deliberately placed this gate after the last compact theorem task, so it would audit a stable claimed scope rather than a moving target. [E22][E23]

The archive audit recorded SHA-256 provenance, inventoried one TeX root and twelve figures, expanded source macros, compiled the article through three passes, and compared the resulting 60-page text with the distributed PDF. The body matched; the observed difference was the arXiv title-page stamp and induced layout. The source-to-Lean trace then classified claims through equation (210) as exact, conditional, constructively strengthened, represented, or outside scope. [E23]

Two findings required correction. A wavelet module header referred to Section 5 instead of Section 3.2. The transmission-invariance boundaries around equations (158)–(173) were not collected in the obligation document. Both were documentation defects. No formula, theorem signature, or proof had to be reopened. [E23]

Meaning of the clean source audit

The audit supports the claim that the finished Lean endpoints have the stated relationship to the source. It does not claim line-by-line formalization of every citation, empirical assertion, or research direction in the paper [1]. A clean conformance result validates the declared boundary; it does not erase that boundary.

The ordering matters historically. The source audit retrospectively confirmed that the early closure, polar, and Hilbert-space decisions were faithful. It cannot be used as evidence that no uncertainty existed while those decisions were first made.

15 What the session record reveals between commits

The commit series contains one accepted state per task. The session trace reveals a different rhythm: propose a small term, compile, inspect the exact goal, patch, and compile again. High-risk analytic tasks were often developed in scratch files and moved only after the public names and proofs stabilized. [E25][E26]

Table 6: Selected scratch histories preserved outside Git.

Scratch path	Patches	What the trace adds to the final account
/private/tmp/ gram-proto.lean	12	Shows the progression from unavailable sum lemmas and coercion failures to the explicit ordered-pair count that preserved $k - 1$.
/private/tmp/ nnls-proto.lean	13	Shows derivative-API attempts, the exact loss-increment pivot, KKT decomposition, and the later convergence layer before permanent modules were created.
ScratchT16.lean	6	Shows the local Fejer/compactness construction used after strict contraction became mathematically impossible.
ScratchT17.lean	33	Shows the entire exact-integration and L^2 route, including unavailable library names, timeouts, Gram calculations, and final move to the wavelet module.
ScratchT18.lean	26	Shows the abandoned elaboration-heavy route and the change to a general kernel-analysis operator plus simple-function density.
ScratchT20.lean	15	Shows exact exponential-kernel integration, algebraic normalization failures, and the final Lipschitz remainder theorem.
ScratchT25.lean	9	Shows why direct structure unfolding was replaced by literal finite-expectation goals for the product law.

The patch count is not a difficulty score. A long diff may be a mechanical rename; a one-line counterexample may force a new theorem architecture. The counts are useful only to refute the

impression that the clean modules appeared in one pass and to locate where the dense proof search occurred.

The trace also retains actions that Git intentionally omits: temporary axiom-audit files, theorem-name probes, failed builds, report renderings, and scratch cleanup. In that sense it is closer to a laboratory notebook than the commit graph. Unlike a laboratory notebook, it is machine-generated and exhaustive only for tool-mediated events. The human-readable session audit selects evidence without publishing raw operational content. [E26]

16 Scale, speed, and what they do not prove

At proof closure the Lean tree contained 31 module files under `IPNPCNS/`, 8,367 Lean lines including the root import module, 439 theorem or lemma declarations, and 172 definitions, structures, classes, or abbreviations. The Git series had 23 commits through source audit and 24 through the scientific report. The permanent mathematical modules ranged from 20 lines for a finite example wrapper to more than 1,000 lines for the exact wavelet development. [E25]

The elapsed commit window is striking. The first feasibility commit is timestamped 17:31:59 local time, the source-audit commit 22:36:58, and the first report commit 23:05:38, all on the same date. The live session itself began earlier at 15:10:21Z and continued through report production. [E25][E26]

Three cautions prevent this from becoming a misleading performance claim.

1. Commit timestamps bound accepted repository states; they do not measure uninterrupted active compute, equivalent human labor, or independent review.
2. The initial estimate concerned experienced human development and included a broad uncertainty margin. It was not a benchmark prediction for an autonomous agent operating in a pinned, cached environment.
3. The final scope is broad but disciplined. It does not formalize every cited background theorem, generic stochastic learning, all convolution kernels, a full intrinsic polyhedral-face theory, or empirical CNS adequacy.

The honest conclusion is therefore not that formal mathematics has become instantaneous. It is that an autonomous agent can compress a large amount of repo-local formal proof work when the environment, task contracts, and feedback loops are favorable. The compression makes evidence discipline more important, not less: without the session trace, clean commits would conceal the dense search; without source and axiom audits, speed could be mistaken for assurance.

17 General lessons for interdisciplinary formalization

The project suggests several lessons that extend beyond this paper.

17.1 Formalize semantic operations before convenient representations

The coordinate-free cone and subspace layers survived every later extension. Pseudoinverse matrices, finite generators, and population circuits could be added as representation theorems. Starting from source notation alone would have coupled the core to missing library infrastructure. [E02][E09][E15]

17.2 Treat quantifiers and closure as part of the result

Changing from arbitrary closed cones to finite polyhedral cones would have made some proofs easier but weakened the headline result. Omitting norm closure would have made several image arguments shorter but false as stated. These are not implementation details; they determine which theorem was verified. [E02][E03][E11]

17.3 Separate exact mathematics, approximation, and empirical realization

Near-isometry implications, random embedding probabilities, learned population accuracy, and biological validity occupy different logical levels. The formalization benefited from different namespaces and interfaces for those levels. A conditional theorem can be strong and useful without converting its premise into an empirical fact. [E04][E05][E23]

17.4 Let counterexamples shape the public API

The rank-deficient kernel obstruction forced prediction and coefficient uniqueness apart. Empty cones forced semantic theorems to carry nonemptiness hypotheses. The nonattainment of an infinite-dimensional infimum led to a lower-bound proof rather than selection of a minimizer. These are improvements in mathematical statement, not setbacks in implementation. [E10][E12][E16]

17.5 Use interfaces as revisitable hypotheses

An interface should name the missing mathematical content narrowly enough that it can later be discharged or specialized. The membrane, exponential occupancy, and convolution interfaces passed this test. Equation (204) also passes it by remaining visible: the project can prove consequences without claiming a training theorem it does not possess. [E05][E18][E20][E21]

17.6 Concrete witnesses can be more honest than generic abstractions

The Volterra theorem did not establish every square-integrable convolution claim. It established one nontrivial causal compact operator with explicit boundary semantics. The product occupancy theorem did not establish biological independence; it established the exact result for a chosen independent experiment. Such witnesses turn vague existence prose into auditable mathematics without overgeneralizing. [E18][E22]

17.7 Proof logs and proof artifacts answer different questions

Git and compiled constants answer what survived. Session patches and diagnostics answer how the proof was found. Wotan and Project Control answer what the work was supposed to establish and why it changed direction. A credible history needs all three, with provenance labels that prevent one from impersonating another. [E26]

17.8 Audit the source after the theorem set stabilizes, but before publication

An early source audit would have chased a moving coverage boundary. A post-publication audit would have been too late to correct theorem or report claims. The selected gate made every material source discrepancy capable of reopening proof work while keeping the traceability exercise finite. [E22][E23]

17.9 Verification is layered assurance

Compilation, absence of admitted proofs, axiom inspection, theorem-signature review, source conformance, and visual report QA answer different questions. None subsumes the others. The project became more credible as these layers accumulated and their failures were themselves recorded. [E23][E24][E26]

17.10 Autonomous speed increases the value of explicit control

When proof production is fast, scope can expand faster than reflective judgment. An obligation register, decision gates, and a rule for empty queues kept the work tied to source-shaped value.

The mechanism did not make the decisions correct by fiat; it made their evidence and uncertainty inspectable.

18 Residual uncertainty and limits of this history

The report is evidence-based, not omniscient.

First, Wotan and Project Control records are contemporaneous but self-authored. They summarize intent and interpretation; they are not independent observations. Git and compiler output corroborate implementation claims, while the source audit corroborates final conformance.

Second, the session trace is unusually complete for tool-mediated editing. It does not record unexpressed cognition, and this report does not expose model-internal reasoning. The checksum fixes the prefix actually audited; later session events are outside that historical snapshot. [E26]

Third, the 38-endpoint axiom audit is a broad representative sample across all mathematical layers, not a printed list for all 439 theorem and lemma declarations. The repository-wide scan separately establishes the absence of admitted proofs, custom axioms, and unsafe declarations. The full build checks dependency closure. [E23]

Fourth, the source audit validates the project's claims about its theorem set. It does not formalize every cited result or resolve the deliberately open stochastic, generic analytic, intrinsic-face, network-stability, or empirical questions. The proof corpus should be described as a substantial verified mathematical layer, not as a complete formal theory of the central nervous system. [E23]

Finally, no independent external Lean reviewer or upstream mathlib maintainer is part of the recorded project. Kernel checking gives strong logical assurance; independent review could still improve names, abstractions, proof maintainability, and the scientific choice of interfaces.

19 Conclusion

The IPNPCNS formalization did not emerge by mechanically transcribing a paper. It emerged by exposing the commitments hidden in mathematical prose and allowing each one to meet a different kind of pressure: type theory, library availability, counterexample, analytic regularity, sampling semantics, biological uncertainty, or source conformance.

The decisive moves were architectural. The Moreau pilot fixed a coordinate-free closed-cone core. Named interfaces kept underspecified learning and approximation claims honest. A kernel counterexample split full-rank and rank-deficient NNLS. Sufficient active certificates replaced an unavailable global face theory. An effective-map inverse supplied Moore–Penrose representation without a rank assumption. Exact Fourier work, finite-rank kernel approximation, Lipschitz analysis, and finite product experiments converted selected analytic and probabilistic interfaces into theorems.

The historical record is as important as the sequence of successes. Git preserves the accepted checkpoints; Wotan and Project Control preserve scope and decisions; the full session trace preserves scratch code and compiler-guided search that the final tree intentionally omits. Their combination makes it possible to say not only that the development now builds, but how its verification boundary was discovered.

The most general lesson is simple: a formalization is credible when its theorem contracts, failed routes, assumptions, trusted base, and source relationship are all visible. Autonomous proof production can be very fast. The evidentiary work that makes the result scientifically legible cannot be skipped.

A Chronological spine of accepted states

The main report is organized by causal arcs. This appendix supplies the minimal chronological spine needed to date those arcs. Times are local commit times on 3 August 2026. A row identifies an accepted Git state, not all work between states.

Table 7: Git checkpoints through the scientific report.

Time	Commit	Checkpoint	Historical role
17:31	e291e52	Feasibility	Paper inventory, Lean probe, empirical boundary, and four inactive continuation ideas.
17:46	eda50fa	Specification	Pinned environment, closure and polar conventions, and exact pilot contract.
17:53	87b1d51	Moreau pilot	Metric projection, Moreau decomposition, cone operations, and double rejection.
18:10	7a29065	Intersection	Three rejection formulas and deterministic projection-error theorem.
18:17	f0db876	Population boundary	Finite NNLS fixed points, explicit approximation interface, and conditional consequences.
18:30	084cf8f	Approximation constants	Polarization and direct sparse Gram bound.
18:38	d5f9d34	Active certificate	Finite sufficient active-face region and population constructor.
19:00	529e111	NNLS optimality	Loss derivative, KKT/minimizer equivalence, and contraction-based convergence.
19:18	560c81e	Subspaces	Intrinsic finite geometry, matrix range identity, and comparison measures.
19:26	48efc13	Spectral NNLS	Full-rank contraction factor and rank-deficient obstruction.
19:36	10baf10	Cone laws and circuits	Exact identities, reflection, primitive circuits, and conditional control.
19:46	d6f610a	Cone comparison	Worst-case similarity, finite-frame proxy, and coverage error.
19:56	a7ecddb	Occupancy	Exact one-message finite experiment and visible approximation boundaries.
20:09	b26dcec	Signal semantics	Finite-window Bochner L^2 , bounded filters, dictionaries, and analytic interfaces.
20:32	764a00f	Moore–Penrose	Effective-map inverse, Penrose equations, uniqueness, and projector renderings.
20:42	94c4bfa	Rank-deficient NNLS	Fejer/compactness convergence to a selected minimizer.
21:15	af5d096	Exact wavelets	Hann-window normalization, orthonormality, examples, and baseline convention.
21:46	08b72d3	Compact Volterra filter	Causal realization, norm bound, finite-rank approximation, and compactness.

Time	Commit	Checkpoint	Historical role
21:54	a0a608e	Projection structure	Positive homogeneity and finite-span reduction.
22:05	3dd6ff9	Membrane remainder	Exact low-pass step and Lipschitz quadratic error.
22:15	123d09a	Occupancy approximation	Signed exponential comparison and explicit mp^2/n error.
22:21	367a1c8	Product overlap	Independent two-message experiment and exact q^2/n expectation.
22:36	fd1361b	Source audit	LaTeX conformance, traceability through equation (210), and final trust checks.
23:05	5a83b13	Scientific report	Separate 24-page non-chronological mathematical account and PDF QA.

B Evidence ledger

The entries below are intentionally compact. Full task evidence appears in `wotan/dev-log/`; steering context appears in `PROJECT-CONTROL.md`; source mapping appears in `SOURCE-AUDIT.md`; and the session extraction method appears in `reports/session-trace-audit.md`.

- E01** Contemporaneous feasibility report and task log, `LEAN-FEASIBILITY.md` and `wotan/dev-log/T-0001.md`, preserved by commit `e291e52`. Supports the initial state, scope distinction, risk inventory, pilot choice, and human-effort forecast.
- E02** `FORMALIZATION-SPEC.md`, Project Control records 001, and commit `eda50fa`. Supports the pinned environment, closure and polar conventions, arbitrary-Hilbert-space target, and explicit non-goals.
- E03** Task log T-0003 and commit `87b1d51`. Supports the Moreau API, pointwise-before-hull proof structure, network-cache recovery, and first assumption/axiom audit.
- E04** Task log T-0004, Project Control reviews 002–003, and commit `7a29065`. Supports the direct four-Moreau intersection architecture, heartbeat adjustment, deterministic near-isometry scope, and decomposition of independent endpoints.
- E05** Task log T-0005, `MODEL-OBLIGATIONS.md`, Project Control review 004, and commit `f0db876`. Supports separation of NNLS mathematics, population assumptions, stochastic claims, and empirical validity.
- E06** Task log T-0006 and commit `084cf8f`. Supports the sharp polarization constant and the direct active quadratic-form proof of the sparse Gram bound.
- E07** Task log T-0007, Project Control reviews 005–006, and commit `d5f9d34`. Supports the sufficient KKT/Moreau active certificate, treatment of boundaries and the empty face, and the unclaimed intrinsic-face converse.
- E08** Task log T-0008, Project Control reviews 006–007, and commit `529e111`. Supports exact loss expansion, KKT/global-minimum equivalence, contraction-based convergence, and the early recognition of the rank-deficient boundary.
- E09** Task log T-0009, Project Control reviews 007–008, and commit `560c81e`. Supports dependency-led expansion, intrinsic subspace results, and deferral of the pseudoinverse representation layer.

- E10** Task log T-0010, Project Control review 009, and commit 48efc13. Supports the full-rank contraction factor, common prediction, and the proved kernel obstruction to strict coefficient contraction.
- E11** Task log T-0011 and commit 10baf10. Supports exact cone laws, reflection and circuit rewrites, explicit control premises, and rejection of an invalid image-membership shortcut.
- E12** Task log T-0012 and commit d6f610a. Supports explicit nonemptiness, nonattainment-safe infimum reasoning, optimistic finite proxies, and coverage-conditional error.
- E13** Task log T-0013, Project Control reviews 011–012, and commit a7ecddb. Supports the exact without-replacement/independent-between-axons experiment, boundary cases, and separate approximation interfaces.
- E14** Task log T-0014, Project Control reviews 012–013, and commit b26dcec. Supports finite-window Bochner L^2 , almost-everywhere semantics, closed image hulls, and the initial analytic interface boundary.
- E15** Task log T-0015, Project Control reviews 013–014, and commit 764a00f. Supports the effective-map Moore–Penrose construction, all-rank scope, four Penrose equations, and dependent projector-equality issue.
- E16** Task log T-0016, Project Control reviews 014–015, and commit 94c4bfa. Supports the rank-deficient Fejer, compactness, cluster-minimizer, and full-sequence convergence proof without a lower spectral bound.
- E17** Task log T-0017, Project Control reviews 015–016, and commit af5d096. Supports exact wavelet constants, local Fourier integration, orthonormality, example coefficients and residuals, and baseline convention.
- E18** Task log T-0018, Project Control reviews 016–017, and commit 08b72d3. Supports the simple-function/rank-one compactness route, causal boundary choice, norm bound, and separation of alternative boundaries.
- E19** Task log T-0019, Project Control reviews 017–018, and commit a0a608e. Supports empty-queue selection, positive homogeneity, span reduction, unrestricted ambient Hilbert space, and dependent-instance rewrite repair.
- E20** Task log T-0020, Project Control reviews 018–019, and commit 3dd6ff9. Supports the exact low-pass integral, Lipschitz premise, selected quadratic constant, and decision not to pursue the sharper half-factor.
- E21** Task log T-0021, Project Control reviews 019–021, and commit 123d09a. Supports constructive discharge of the exponential occupancy error interface and separate invalid regimes.
- E22** Task log T-0025, Project Control reviews 021–022, and commit 367a1c8. Supports the Cartesian-product two-message experiment, exact overlap expectation, and proof-phase closure decision.
- E23** SOURCE-AUDIT.md, task log T-0022, Project Control reviews 022–023, and commit fd1361b. Supports archive provenance, source build and comparison, equation-level traceability, documentation corrections, final 2,960-job build, gap scan, and 38-endpoint axiom audit.
- E24** Task log T-0023, Project Control review 024, and commit 5a83b13. Supports the separate scientific-report role, 24-page artifact, and render-correct QA loop.
- E25** Git log and current Lean-tree measurement. Supports the accepted checkpoint order and timestamps, 31 module files, 8,367 Lean lines, 439 theorem or lemma declarations, and 172 definition-like declarations. Timestamps do not measure human-equivalent effort.
- E26** reports/session-trace-audit.md and the main Codex session prefix with SHA-256 80e1b5977d4ff07fbfc696f319eb2a31db33d57636464e20b81d1063ba763ba5. Supports the scratch-file reconstruction, 427 successful patch events, 174 changes over 33 temporary or audit files, compiler-feedback chains, and report QA history.

References

- [1] M. N. P. Nilsson. Information processing by neuron populations in the central nervous system: A theory of the mathematical structure of data and operations. arXiv:2309.02332v3, 2026.
- [2] J.-J. Moreau. Decomposition orthogonale d'un espace hilbertien selon deux cones mutuellement polaires. *C. R. Acad. Sci. Paris*, 255:238–240, 1962.
- [3] C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. SIAM, Philadelphia, 1995.
- [4] H. H. Bauschke and P. L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. Springer, second edition, 2017.
- [5] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The Lean theorem prover (system description). In *Automated Deduction – CADE-25*, pages 378–388. Springer, 2015.
- [6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.
- [7] I. Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery*. Cambridge University Press, 1976.